

Factory Method & Abstract Factory

Md Saeed Siddik

Assistant Professor,
Institute of IT, University of Dhaka

2

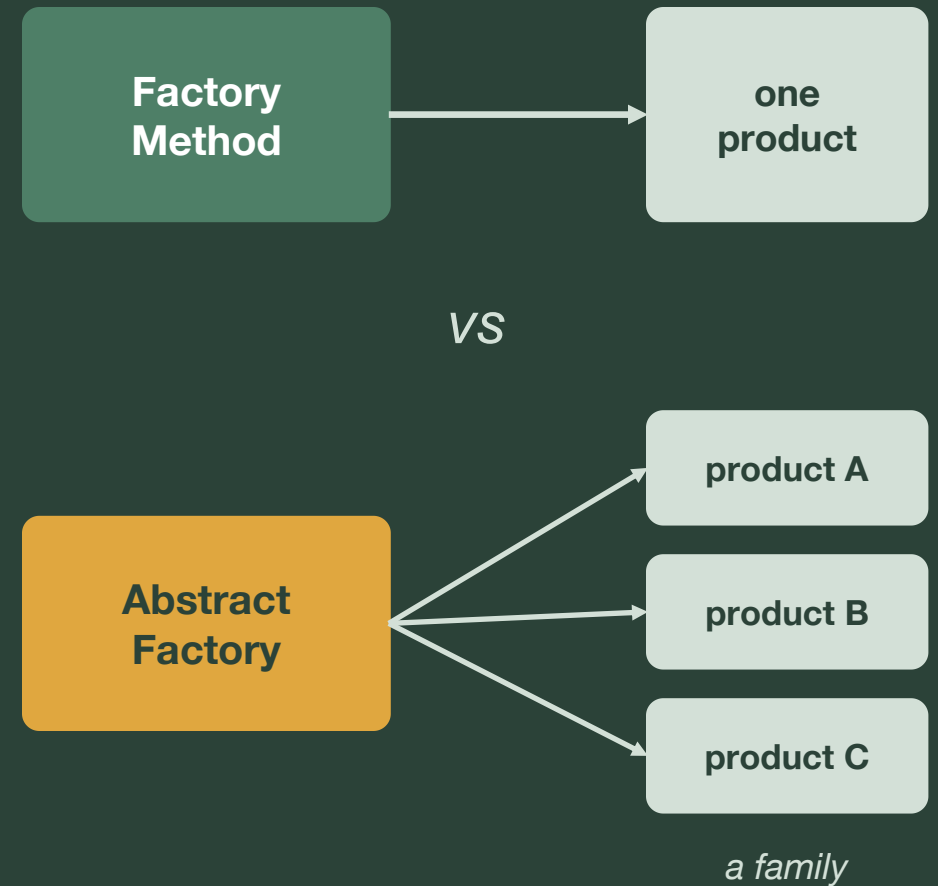
patterns

2

case studies

7

Java demos



What we will cover

Two patterns · two case studies · one comparison



01

Vocabulary

Six things people call “factory”, and which two are patterns



02

Factory Method

Intent, cases, problematic code, refactoring, results



03

Abstract Factory

Families of related objects, the same journey



04

Side by side

Structure, the matrix view and a comparison table



05

Choosing

A decision guide and how one evolves into the other



06

Practice

A class activity and the key takeaways

Six things people call “factory”

Refactoring.Guru's factory comparison, only the last two are design patterns

Factory

An ambiguous word: any function, method or class that produces something.

Creation method

Any method that creates objects, often just a wrapper around a constructor.

Static creation method

A creation method declared static: an alternative constructor, or a way to return cached objects.

Simple factory

One method with a big conditional that picks the class to instantiate. A stepping stone, not a GoF pattern.

DESIGN PATTERN

Factory Method

A creator declares a creation method; subclasses decide which product it returns.

DESIGN PATTERN

Abstract Factory

An interface for creating whole families of related products without naming concrete classes.

01

Factory Method

Let subclasses decide which class to instantiate

Creator

Product

Concrete Creator

Concrete Product



Intent and structure

INTENT

Provide an interface for creating objects in a superclass, but let subclasses alter the type of objects that will be created.

ALSO KNOWN AS

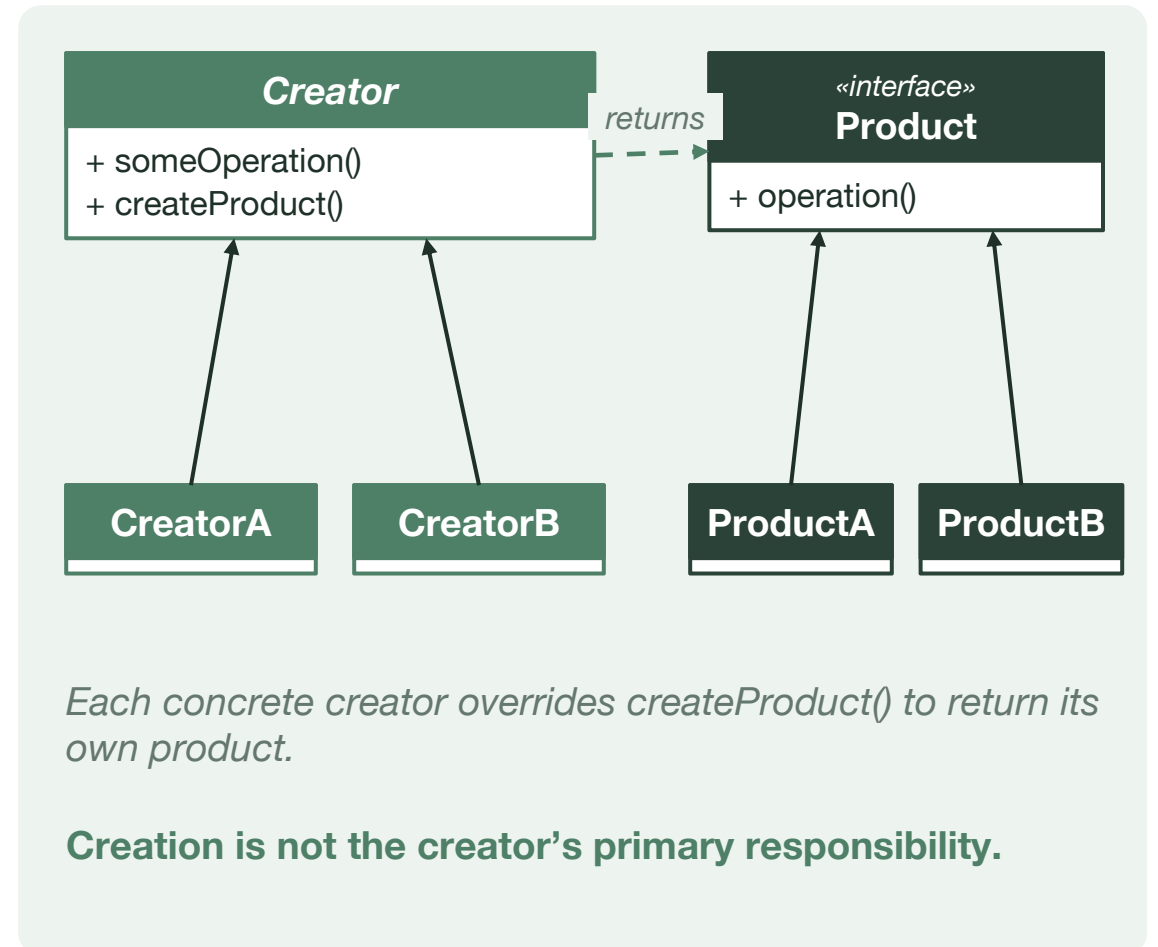
Virtual Constructor.

ANALOGY

A software company has a training department, yet its main job is writing code. A creator's main job is not creating products either, it has business logic that uses them.

THE RULE

All products must share a common interface.



Where Factory Method is used



Payment systems

Create card, UPI or wallet processors depending on how the customer pays.



Game development

Create different enemies, weapons or items depending on level or environment.



Document processing

Create PDF, Word or text document objects for the same editor logic.



Notification systems

Create Email, SMS or Push notifications based on the required channel.

In the JDK: `Collection.iterator()`, every collection returns the Iterator that fits it.

When to use it

1

The exact types are not known beforehand

Separate construction from use. To add a new product, add a new creator subclass and override the factory method, nothing else changes.

2

Users should be able to extend your framework

Reduce the code that constructs components to one factory method. Someone overrides it (e.g., `createButton()` returns a `RoundButton`) and the framework uses their component.

3

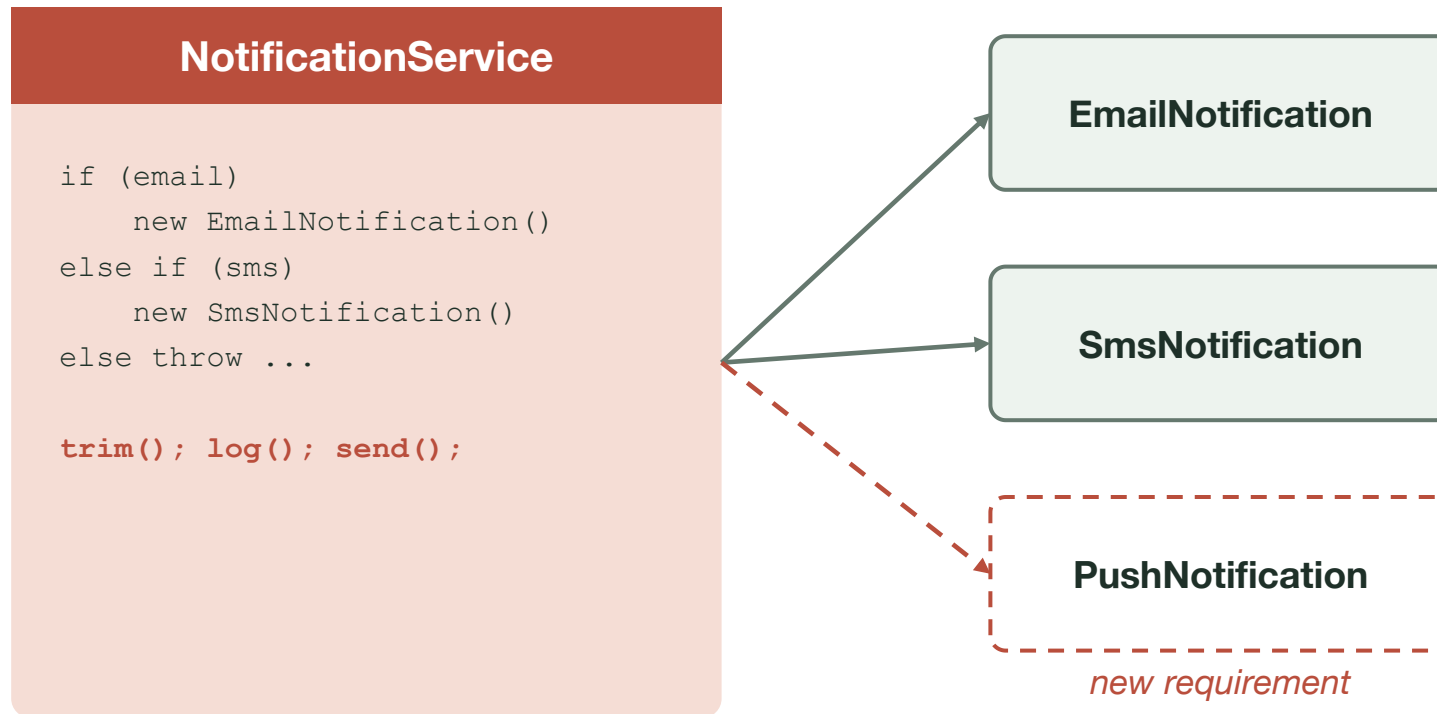
You want to reuse existing objects

A constructor must return a new object. A factory method can look in a pool or cache first, useful for large objects like database connections.

PROBLEMATIC CASE

A notification service that must grow

The service picks the class itself, and creation is mixed with business logic



WHAT GOES WRONG



Tight coupling

The service depends on every concrete notification class.



Hard to extend

A new channel means editing shipped, tested code.



Growing conditionals

Every type adds another else-if, and creation and logic stay tangled.

The problematic code (1/2)

Notifications.java

```
interface Notification {
    void send(String to, String msg);
}

class EmailNotification implements Notification {
    public void send(String to, String msg) {
        System.out.printf("EMAIL -> %s: %s%n", to, msg);
    }
}

class SmsNotification implements Notification {
    public void send(String to, String msg) {
        System.out.printf("SMS -> %s: %s%n", to, msg);
    }
}

class PushNotification implements Notification {
    public void send(String to, String msg) {
        System.out.printf("PUSH -> %s: %s%n", to, msg);
    }
}
```

NotificationService.java

```
public class NotificationService {

    public void notifyUser(String channel, String to,
                          String message) {
        // creation code ...
        Notification n;
        if (channel.equals("email")) {
            n = new EmailNotification();
        } else if (channel.equals("sms")) {
            n = new SmsNotification();
        } else {
            throw new IllegalArgumentException(
                "Unknown channel: " + channel);
        }

        // ... mixed with business logic
        String text = message.trim();
        System.out.println("[log] via " + channel);
        n.send(to, text);
    }
}
```

Creation and logic live together: the if/else is the problem.

The problematic code (2/2) and its run

ProblemDemo.java

```
public class ProblemDemo {  
    public static void main(String[] args) {  
        NotificationService service = new NotificationService();  
        String msg = "Order shipped";  
  
        service.notifyUser("email", "ana@shop.com", msg);  
        service.notifyUser("sms", "555-0101", msg);  
  
        try { // marketing asks for push notifications...  
            service.notifyUser("push", "device-42", msg);  
        } catch (IllegalArgumentException e) {  
            System.out.println("ERROR: " + e.getMessage());  
        }  
    }  
}
```



```
$ java ProblemDemo  
[log] via email  
EMAIL -> ana@shop.com: Order shipped  
[log] via sms  
SMS -> 555-0101: Order shipped  
ERROR: Unknown channel: push
```

PushNotification already exists, but the only way to use it is to edit NotificationService. Every new channel means another else-if in code that is already shipped and tested.

The refactoring recipe

Refactoring.Guru's steps, applied to the notification service

1 Make all products follow one interface

```
interface Notification { void send(...); }
```

2 Add an (empty) factory method to the creator

```
Notification createNotification() { return null; }
```

3 Replace product constructor calls with the factory method

```
Notification n = createNotification();
```

4 Create a creator subclass per product; override the method

```
class SmsService extends NotificationService { ... }
```

5 Make the base factory method abstract

```
protected abstract Notification createNotification();
```

A stepping stone: the simple factory

NotificationFactory.java

```
public class NotificationFactory {

    public static Notification create(String channel) {
        switch (channel) {
            case "email": return new EmailNotification();
            case "sms":   return new SmsNotification();
            case "push":  return new PushNotification(); // new
            default:
                throw new IllegalArgumentException(
                    "Unknown channel: " + channel);
        }
    }
}
```



Creation code has left the service.



Every new type still edits the switch.



It is static: no subclass can change what it creates.

Not a GoF pattern. Usually an intermediate step towards Factory Method or Abstract Factory.



```
$ java SimpleFactoryDemo
[log] via email ... EMAIL -> ana@shop.com
[log] via sms   ... SMS   -> 555-0101
[log] via push  ... PUSH  -> device-42
```

Refactored: Factory Method (1/2)

NotificationService.java (the creator)

```
public abstract class NotificationService {

    // the factory method: subclasses decide the product
    protected abstract Notification createNotification();

    public void notifyUser(String to, String message) {
        Notification n = createNotification();
        String text = message.trim(); // business logic
        System.out.println("[log] "
            + getClass().getSimpleName());
        n.send(to, text);
    }
}
```

Services.java + PushService.java

```
class EmailService extends NotificationService {
    @Override
    protected Notification createNotification() {
        return new EmailNotification();
    }
}

class SmsService extends NotificationService {
    @Override
    protected Notification createNotification() {
        return new SmsNotification();
    }
}

// NEW: added later - no existing file was edited
class PushService extends NotificationService {
    @Override
    protected Notification createNotification() {
        return new PushNotification();
    }
}
```

The business logic is written once. Subclasses only choose the product; notifyUser() never mentions a concrete class.

Refactored: Factory Method (2/2)

FactoryMethodDemo.java

```
public class FactoryMethodDemo {  
    public static void main(String[] args) {  
        NotificationService[] services = {  
            new EmailService(),  
            new SmsService(),  
            new PushService()  
        };  
        String[] to = { "ana@shop.com", "555-0101", "device-42" };  
        String msg = "  Order shipped ";  
  
        for (int i = 0; i < services.length; i++) {  
            services[i].notifyUser(to[i], msg);  
        }  
    }  
}
```



```
$ java FactoryMethodDemo  
[log] EmailService  
EMAIL -> ana@shop.com: Order shipped  
[log] SmsService  
SMS -> 555-0101: Order shipped  
[log] PushService  
PUSH -> device-42: Order shipped
```

Adding push = one new class. NotificationService, EmailService and SmsService: untouched.

The choice of creator is made once, at start-up or from configuration.

Adding a channel: before vs after

Before: edit existing code

NotificationService.java

```
    } else if (channel.equals("sms")) {  
        n = new SmsNotification();  
+   } else if (channel.equals("push")) {  
+       n = new PushNotification();  
    } else {  
        throw new IllegalArgumentException(...);
```

After: add a new file

PushService.java (new)

```
+ class PushService extends NotificationService {  
+     @Override  
+     protected Notification createNotification() {  
+         return new PushNotification();  
+     }  
+ }
```

1

existing file edited
and re-tested

0

existing files edited
Open/Closed Principle in action

Advantages and limitations



Advantages

- ✓ No tight coupling between creator and concrete products
- ✓ Single Responsibility: creation code in one place
- ✓ Open/Closed: new products without breaking clients
- ✓ Business logic in the creator is reused for every product



Limitations

- ✗ Many new subclasses, best added to an existing creator hierarchy
- ✗ Can be overkill for small, simple applications
- ✗ A static creation method cannot be overridden, it defeats the pattern
- ✗ All products need a common interface

Related patterns: Factory Method specializes Template Method; many designs start here and evolve towards Abstract Factory, Prototype or Builder.

02

Abstract Factory

Produce families of related objects that always match

Abstract Factory

Concrete Factory

Abstract Product

Concrete Product



Intent and the product matrix

INTENT

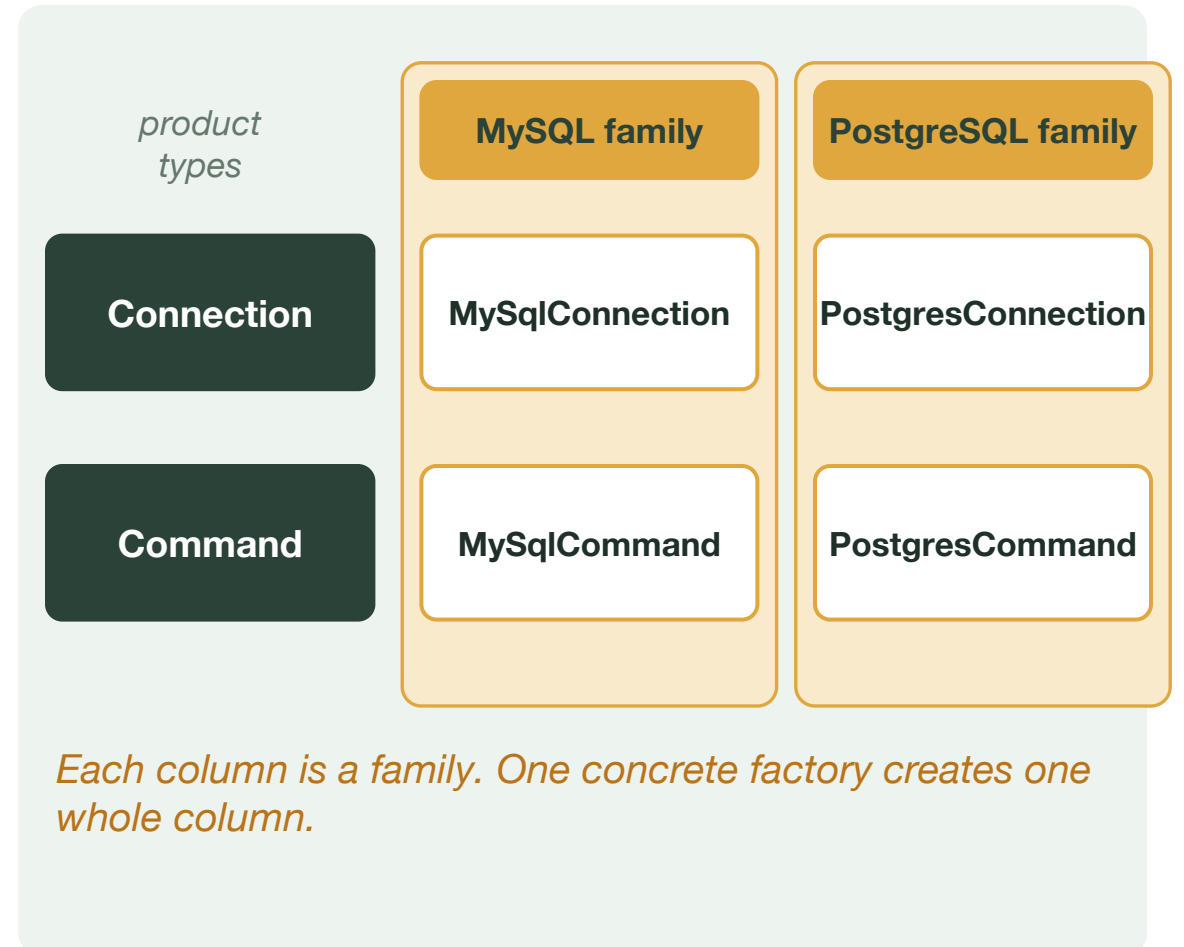
Provide an interface for creating families of related or dependent objects without specifying their concrete classes.

ANALOGY

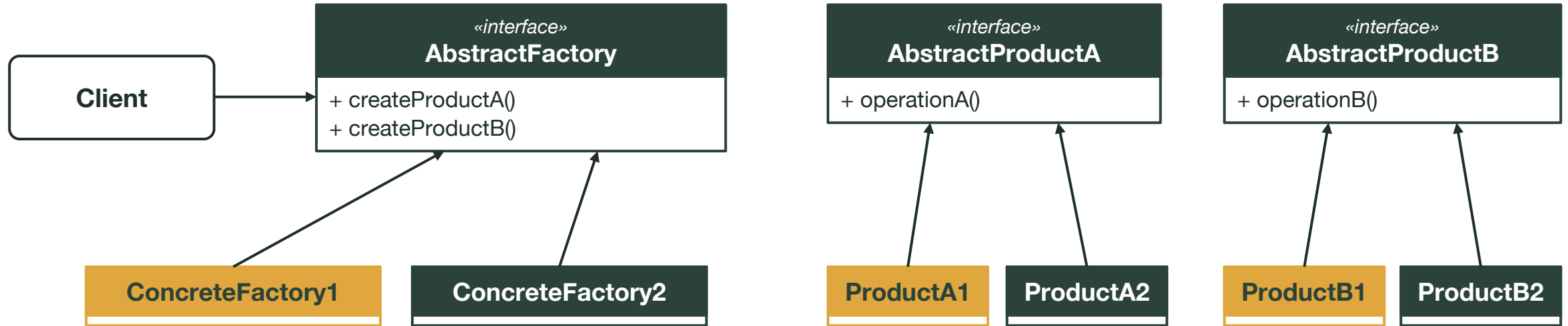
Sheet-metal stamping equipment makes car body parts. Swap the stamping dies and the same machine produces the parts of a different model.

KEY IDEA

Clients never call new. The concrete factory class appears once, where it is instantiated.



Structure and components



Same colour = same family: Factory1 creates A1 + B1, Factory2 creates A2 + B2.

Abstract Factory

One creation method per product type

Concrete Factories

One per variant (family)

Abstract Products

One interface per product type

Concrete Products

Implementations, grouped by variant

Client

Uses only the abstract types

Where and when to use it



UI themes

Light or Dark theme, each with its own Button and TextBox.



Cloud platforms

AWS, Azure or Google Cloud: storage, compute, networking.



Database systems

SQL or NoSQL: connections, commands and queries that match.



Portable apps

Windowing system, OS and other platform dependencies.

USE IT WHEN

Your code must work with several product families

You do not want to depend on their concrete classes, they may be unknown today or added later. Platform independence is the current source of pain.

USE IT WHEN

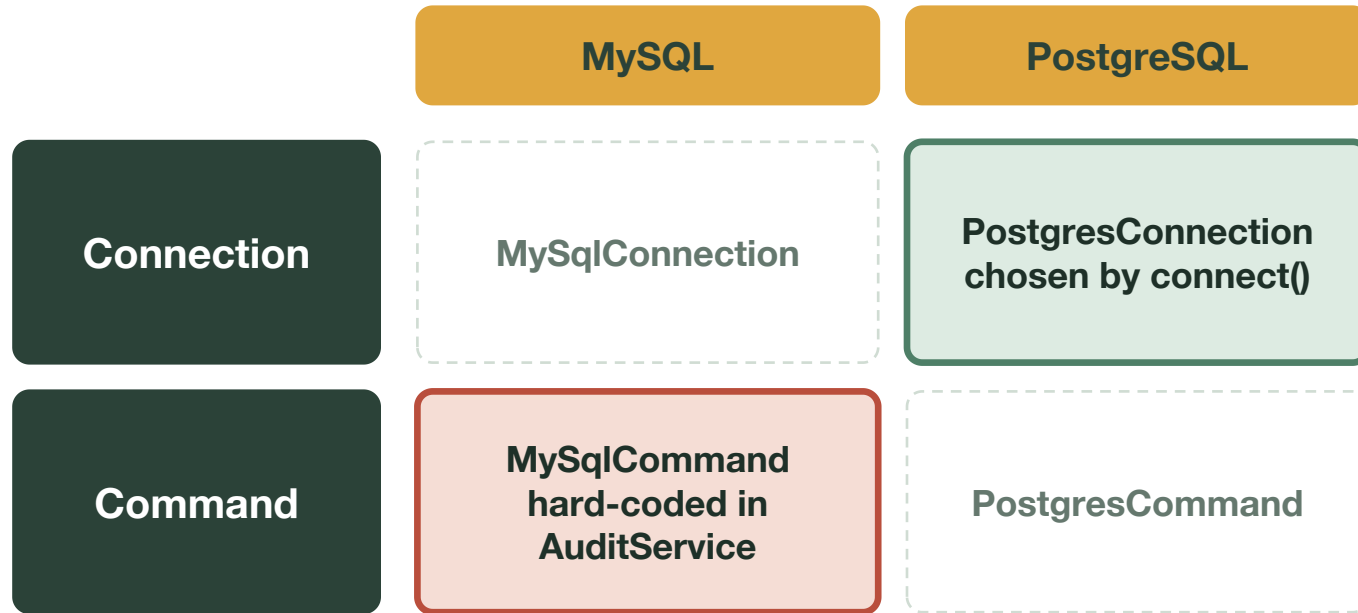
A class has a set of Factory Methods that blur its job

Extract them into a stand-alone factory class (or a full Abstract Factory) so each class is responsible for one thing.

PROBLEMATIC CASE

Objects from different families get mixed

A data-access layer that must support MySQL and PostgreSQL



WHAT GOES WRONG



Mismatched families

Nothing stops objects from different families being combined.



Scattered creation

Each client has its own if/else or hard-coded new.



Fragile growth

A new database means hunting for every creation site.



Two creation sites picked two different columns, an incompatible pair: a PostgreSQL connection with a MySQL command.

The products (unchanged by the refactoring)

Commands.java

```
interface Command {
    void execute(Connection c, String sql);
    default void requireDialect(Connection c, String want) {
        if (!c.dialect().equals(want))
            throw new IllegalStateException(
                getClass().getSimpleName()
                + " cannot run on '" + c.dialect() + "'");
    }
}

class MySQLCommand implements Command {
    public void execute(Connection c, String sql) {
        requireDialect(c, "mysql");
        System.out.println("[mysql] executing: " + sql);
    }
}

class PostgreSQLCommand implements Command {
    public void execute(Connection c, String sql) {
        requireDialect(c, "postgres");
        System.out.println("[postgres] executing: " + sql);
    }
}
```

Connections.java

```
interface Connection {
    String dialect();
    void open();
}

class MySQLConnection implements Connection {
    public String dialect() { return "mysql"; }
    public void open() {
        System.out.println(
            "[mysql] connection opened");
    }
}

class PostgreSQLConnection implements Connection {
    public String dialect() { return "postgres"; }
    public void open() {
        System.out.println(
            "[postgres] connection opened");
    }
}
```

Each Command insists on its own dialect.

The problematic clients and their run

ProblemDemo.java

```
public class ProblemDemo {
    // Connections are created here ...
    static Connection connect(String db) {
        if (db.equals("mysql"))
            return new MySqlConnection();
        if (db.equals("postgres"))
            return new PostgresConnection();
        throw new IllegalArgumentException("Unknown db: " + db);
    }

    public static void main(String[] args) {
        String[] dbs = { "mysql", "postgres" };
        for (String db : dbs) {
            System.out.println("--- " + db + " ---");
            try {
                Connection conn = connect(db);
                conn.open();
                new AuditService().log(conn);
            } catch (RuntimeException e) {
                System.out.println("ERROR: " + e.getMessage());
            }
        }
    }
}
```

AuditService.java

```
class AuditService {
    // ... and commands are created here
    // (written when only MySQL existed)
    void log(Connection conn) {
        Command cmd = new MySqlCommand();
        cmd.execute(conn, "INSERT INTO audit ...");
    }
}
```

```
$ java ProblemDemo
--- mysql ---
[mysql] connection opened
[mysql] executing: INSERT ...
--- postgres ---
[postgres] connection opened
ERROR: MySqlCommand cannot run on 'postgres'
```

The refactoring recipe

1 Map the matrix: variants x product types

```
// MySQL, Postgres x Connection, Command
```

2 Declare an interface for every product type

```
interface Connection { ... }
interface Command { ... }
```

3 Declare the factory: one creation method per product

```
Connection createConnection();
Command createCommand();
```

4 Write one concrete factory per variant

```
class MySQLFactory implements DatabaseFactory
```

5 Create the factory once and pass it to clients

```
new AuditService(factoryFor(db))
```

6 Replace every direct new in clients

```
factory.createCommand() // was: new MySQLCommand()
```

Refactored: Abstract Factory (1/2)

DatabaseFactory.java

```
interface DatabaseFactory {
    Connection createConnection();
    Command createCommand();
}

class MySQLFactory implements DatabaseFactory {
    public Connection createConnection() {
        return new MySQLConnection();
    }
    public Command createCommand() {
        return new MySQLCommand();
    }
}

class PostgresFactory implements DatabaseFactory {
    public Connection createConnection() {
        return new PostgresConnection();
    }
    public Command createCommand() {
        return new PostgresCommand();
    }
}
```

AuditService.java

```
class AuditService {
    private final DatabaseFactory factory;

    AuditService(DatabaseFactory factory) {
        this.factory = factory;
    }

    void log() {
        Connection conn = factory.createConnection();
        // products from ONE factory always match
        Command cmd = factory.createCommand();
        conn.open();
        cmd.execute(conn, "INSERT INTO audit ...");
    }
}
```

AuditService names no concrete class. It only knows DatabaseFactory, Connection and Command.

Refactored: Abstract Factory (2/2)

AbstractFactoryDemo.java

```
public class AbstractFactoryDemo {

    // the ONLY place that knows the concrete factories
    static DatabaseFactory factoryFor(String db) {
        switch (db) {
            case "mysql":    return new MySqlFactory();
            case "postgres": return new PostgresFactory();
            default:
                throw new IllegalArgumentException(
                    "Unknown db: " + db);
        }
    }

    public static void main(String[] args) {
        String[] dbs = { "mysql", "postgres" };

        for (String db : dbs) {
            System.out.println("--- " + db + " ---");
            new AuditService(factoryFor(db)).log();
        }
    }
}
```



```
$ java AbstractFactoryDemo
--- mysql ---
[mysql] connection opened
[mysql] executing: INSERT ...
--- postgres ---
[postgres] connection opened
[postgres] executing: INSERT ...
```

A mismatch is impossible: one factory, one family.

The concrete factory class appears once.

Adding a whole new family

```

SQLiteFamily.java (new)
// NEW FAMILY: three new classes, no existing class edited
class SQLiteConnection implements Connection {
    public String dialect() { return "sqlite"; }
    public void open() {
        System.out.println("[sqlite] file opened");
    }
}

class SQLiteCommand implements Command {
    public void execute(Connection c, String sql) {
        requireDialect(c, "sqlite");
        System.out.println("[sqlite] executing: " + sql);
    }
}

class SQLiteFactory implements DatabaseFactory {
    public Connection createConnection() {
        return new SQLiteConnection();
    }
    public Command createCommand() {
        return new SQLiteCommand();
    }
}

```

THE MATRIX GAINS A COLUMN

	MySQL	Postgres	SQLite NEW
Connection	done	done	new
Command	done	done	new

```
case "sqlite": return new SQLiteFactory(); // +1
```

```

$ java NewFamilyDemo (last lines)
[sqlite] file opened
[sqlite] executing: INSERT INTO audit ...

```

Advantages and limitations



Advantages

- ✓ Products from one factory are guaranteed compatible
- ✓ No tight coupling between concrete products and clients
- ✓ Single Responsibility: creation code in one place
- ✓ Open/Closed: new variants without breaking clients



Limitations

- ✗ Many new interfaces and classes
- ✗ Overkill for small projects
- ✗ A new product type changes the abstract factory and every concrete factory
- ✗ Something must choose the concrete factory at start-up

Related patterns: often built from Factory Methods (or Prototypes); factories are routinely Singletons; Builder returns its product after several steps, Abstract Factory returns it immediately.

03

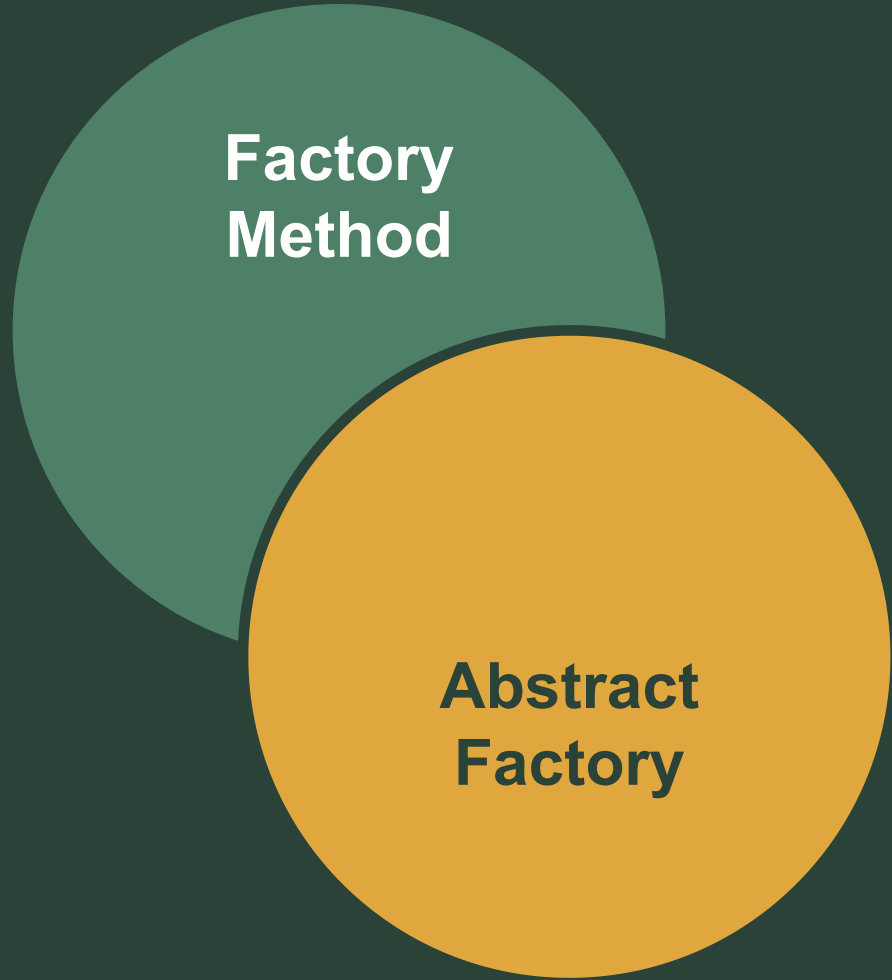
Side by side

Same goal, different scope, mechanism and trade-offs

Comparison table

Matrix view

Decision guide



Factory
Method

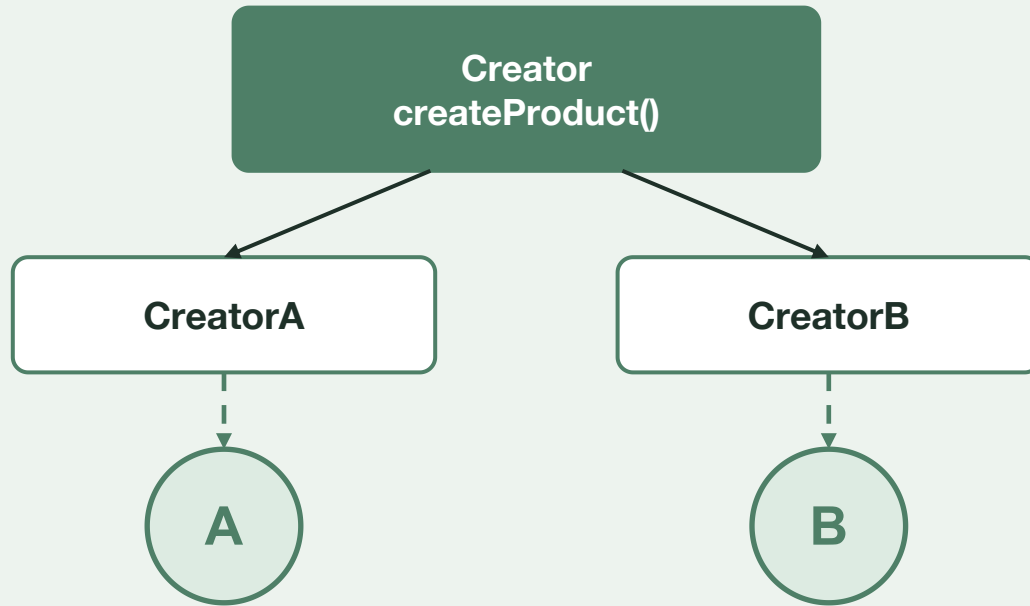
Abstract
Factory

Factory Method vs Abstract Factory

Aspect	Factory Method	Abstract Factory
Intent	Let subclasses decide which class of one product to instantiate	Create families of related products without naming concrete classes
Creates	One product per creator	Several related products (one per product type)
Mechanism	Inheritance: a subclass overrides the factory method	Composition: a factory object is passed to the client
Client works with	The creator, whose logic calls createProduct()	The factory, which it asks for each product
Consistency	Nothing ties different products together	Products from one factory are guaranteed to match
New variant	New creator subclass (plus product class)	New concrete factory plus one class per product type
New product type	Just another creator: easy	Change the interface and every factory: hard

Structure at a glance

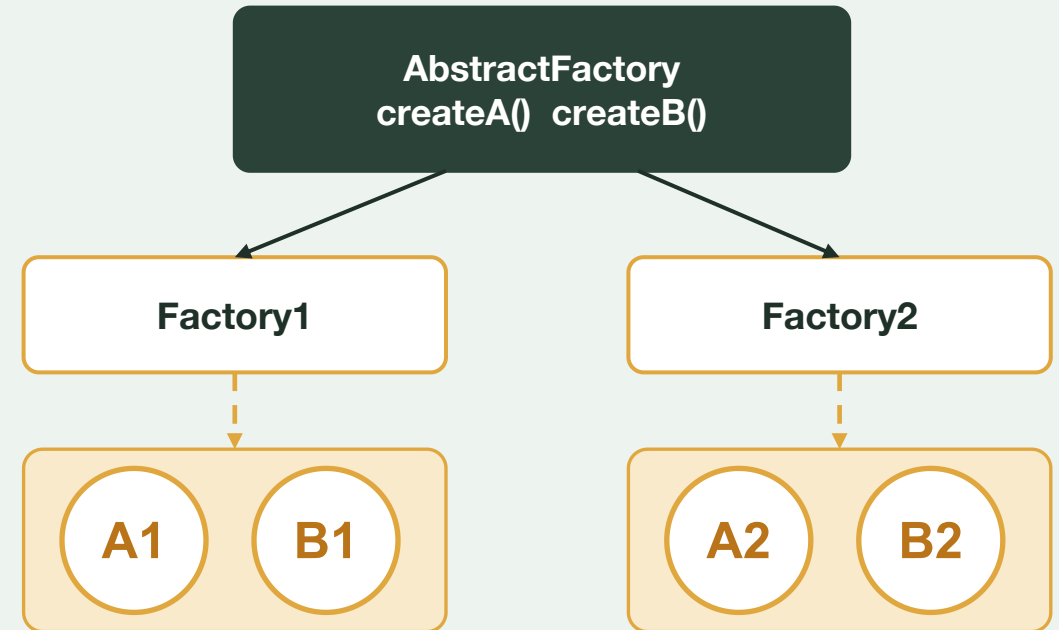
Factory Method



one creator subclass → one product

Inheritance: a subclass overrides one method.

Abstract Factory



one factory → a family of products

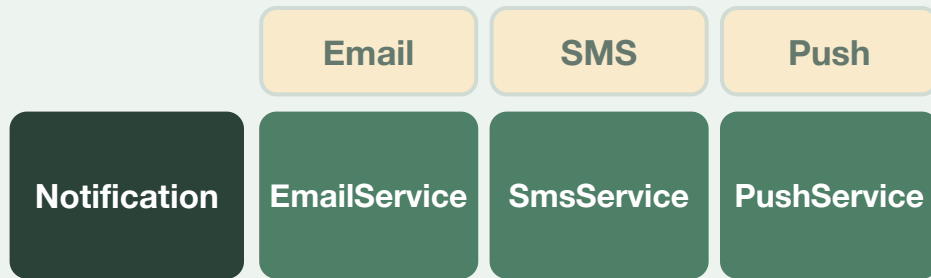
Composition: a factory object is passed in.

DIFFERENCES

The matrix view

Variants across, product types down

Factory Method: one row



Each cell is one creator subclass.

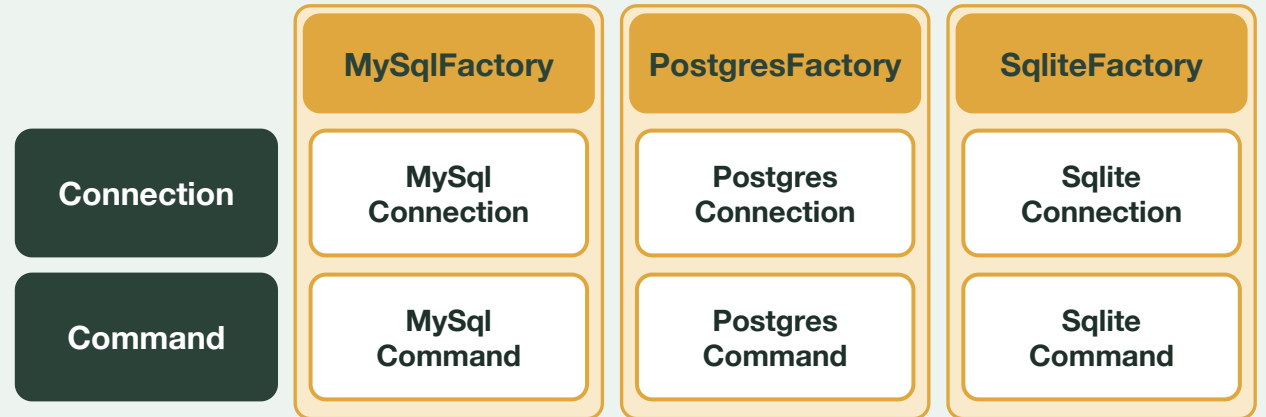
+ new variant

add a cell → one new subclass

+ new product type

a separate creator hierarchy

Abstract Factory: the whole grid

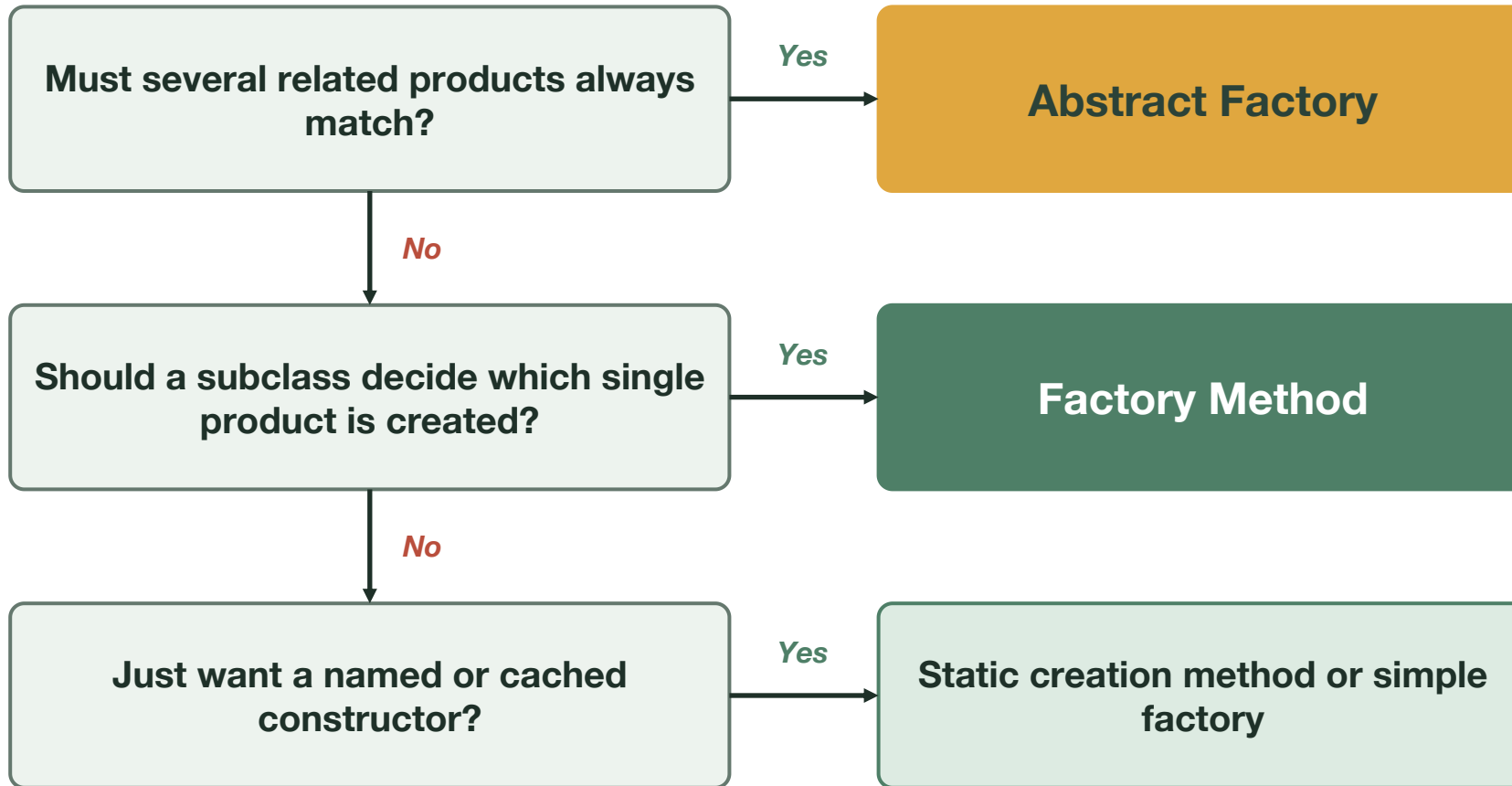


Each factory creates one whole column.

+ new variant add a column: one new factory (easy)

+ new product type add a row: change the interface and every factory (hard)

Decision guide: which one?



No → a plain new is fine.

EVOLUTION

Many designs start with Factory Method, simpler and customizable through subclasses, and evolve towards Abstract Factory, Prototype or Builder as more flexibility is needed.

Abstract Factory classes are often built from a set of Factory Methods.

Class activity: refactor the report exporter

Work in pairs · 25 minutes · starter code in 07_activity_starter

```
ReportExporter.java (starter)
public class ReportExporter {

    public String export(String format, String data) {
        Renderer renderer;
        Header header;
        if (format.equals("pdf")) {
            renderer = new PdfRenderer();
            header = new PdfHeader();
        } else if (format.equals("html")) {
            renderer = new HtmlRenderer();
            header = new HtmlHeader();
        } else {
            throw new IllegalArgumentException(format);
        }
        return header.render() + renderer.render(data);
    }
}
```



```
$ java StarterDemo
PDF-HEADER|PDF[Q3 sales]
```

1

Spot it Which lines change for every new format?

2

Map it Draw the matrix: formats × product types (Renderer, Header).

3

Choose Factory Method or Abstract Factory, and why?

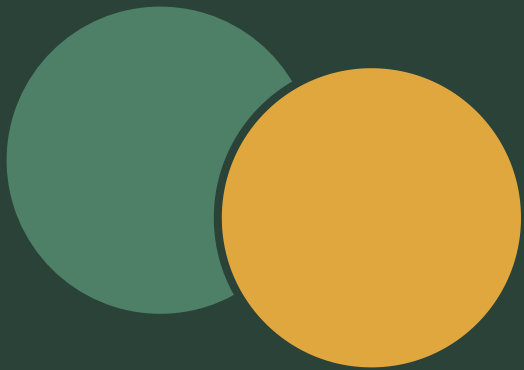
4

Refactor Apply it, then add “markdown” without editing existing classes.

5

Discuss What changes if we add a Footer product type?

Key takeaways



1

Both patterns keep client code away from concrete classes.

2

Factory Method: one product, chosen by a subclass (inheritance).

3

Abstract Factory: a family of matching products, chosen by a factory object (composition).

4

A simple factory or static creation method is a useful first step, but not either pattern.

5

Adding variants is easy in both; adding a new product type is hard for Abstract Factory.

Sources & companion code



Refactoring.Guru

Problem, solution, how-to steps, pros and cons.

[Factory Method](#)



Refactoring.Guru

Product families, structure and applicability.

[Abstract Factory](#)



GeeksforGeeks

Components, vehicle example and applications.

[Factory Method](#)



GeeksforGeeks

Factory of factories, car-region example.

[Abstract Factory](#)



SourceMaking

Intent, checklist and rules of thumb.

[Abstract Factory](#)



COMPANION CODE (ZIP)

01_fm_problem · 02_simple_factory · 03_factory_method
04_af_problem · 05_abstract_factory · 06_new_family
07_activity_starter · run_all.sh (JDK 11+)

ALSO
READ · [REFACTORING.GURU](#)
[Factory Comparison](#)